

Assignment 6

Aim

Write a program to create, traverse (pre, in and post) and display a binary search tree and also insert, delete and search node in it. Create a tree of 10 nodes and perform all other operations

Code

```
#include <stdio.h>
#include <stdlib.h>

typedef
struct Node {
    int data; struct Node *left; struct Node *right;
} Node;

Node* createNode insert(Node* root, int data) {
    if (root == NULL) {
        Node* newNode = (Node*) malloc(sizeof(Node));
        newNode->data = data;
        newNode->left = NULL;
        newNode->right = NULL;
        return newNode;
    }
    if (data < root->data) {
        root->left = insert(root->left, data);
    }
    else if (data > root->data) {
        root->right = insert(root->right, data);
    }
    return root;
}
```

```
void preorder(Node *root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}
```

```
void inorder(Node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
```

```
void postorder(Node *root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}
```

```
Node *search(Node *root, int key) {
    if (root == NULL || root->data == key) {
        return root;
    }
    if (key < root->data) {
        return search(root->left, key);
    }
    return search(root->right, key);
}
```

```
Node* findMin(Node* root) {
    Node* current = root;
    while (current != NULL && current->left != NULL) {
        current = current->left;
    }
    return current;
}
```

```
Node* deleteNode(Node* root, int key) {
    if (root == NULL) {
        return root;
    }
    if (key < root->data) {
        root->left = deleteNode(root->left, key);
    }
    else if (key > root->data) {
        root->right = deleteNode(root->right, key);
    }
    else {
        if (root->left == NULL && root->right == NULL) {
            free(root);
            return NULL;
        }
        else if (root->left == NULL) {
            Node* temp = root->right;
            free(root);
            return temp;
        }
        else if (root->right == NULL) {
            Node* temp = root->left;
            free(root);
            return temp;
        }
        else {

```

```
Node* temp = findMin(root->right);  
root->data = temp->data;  
root->right = deleteNode(root->right,  
temp->data);
```

```
}
```

```
}
```

```
return root;
```

```
}
```

```
void display(Node* root) {  
    printf("In Preorder Traversal: ");  
    preorder(root);  
    printf("In Inorder Traversal: ");  
    inorder(root);  
    printf("In Postorder Traversal: ");  
    postorder(root);  
    printf("\n");  
}
```

```
void main() {  
    Node* root = NULL;  
    Node* result;  
    int values[10] = {50, 30, 70, 20, 40, 60, 80,  
10, 35, 90};  
    printf("Creating binary tree with nodes  
{50, 30, 70, 20, 40, 60, 80, 10, 35, 90}\n");  
    for (int i = 0; i < 10; i++) {  
        root = insert(root, values[i]);  
    }  
    printf("In Original Binary Search Tree: ");  
    display(root);
```

```
int key = 40;
printf("In Searching for node %d ... \n", key);
result = search(root, key);
if (result != NULL) {
    printf("Node %d found in the BST. \n", key);
} else {
    printf("Node %d for not found in the  
BST. \n", key);
}

printf("In inserting node 45 ... \n");
root = insert(root, 45);
printf("BST after insertion:");
display(root);
printf printf("In deleting node 30 ... \n");
root = deleteNode(root, 30);
printf("BST after deletion:");
display(root);
}
```

Creating Binary Search Tree with nodes {50, 30, 70, 20, 40, 60, 80, 10, 35, 90}

Original Binary Search Tree:

Preorder Traversal : 50 30 20 10 40 35 70 60 80 90

Inorder Traversal : 10 20 30 35 40 50 60 70 80 90

Postorder Traversal : 10 20 35 40 30 60 90 80 70 50

Searching for node 40...

Node 40 found in the BST.

Inserting node 45...

BST after insertion:

Preorder Traversal : 50 30 20 10 40 35 45 70 60 80 90

Inorder Traversal : 10 20 30 35 40 45 50 60 70 80 90

Postorder Traversal : 10 20 35 45 40 30 60 90 80 70 50

Deleting node 30...

BST after deletion:

Preorder Traversal : 50 35 20 10 40 45 70 60 80 90

Inorder Traversal : 10 20 35 40 45 50 60 70 80 90

Postorder Traversal : 10 20 45 40 35 60 90 80 70 50